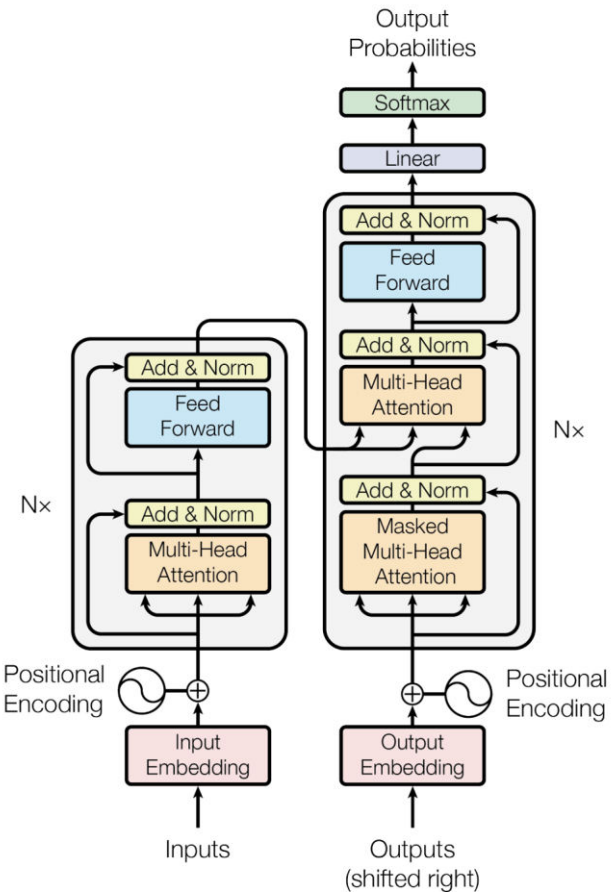
9. Transformers - Architecture

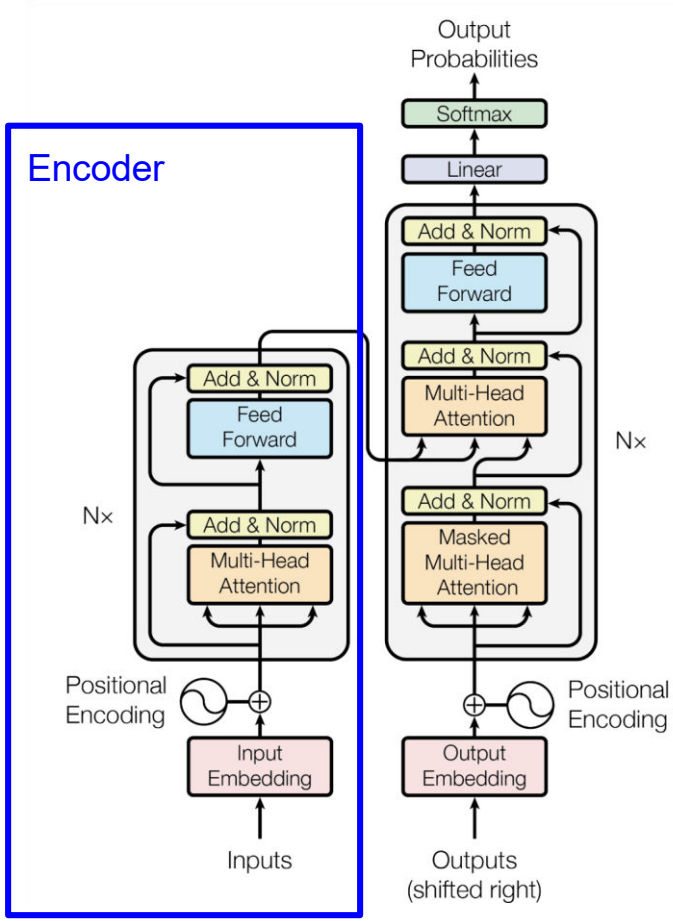
with intricate nuances

Key learning:
Tokenization, Byte-Pair Encoding

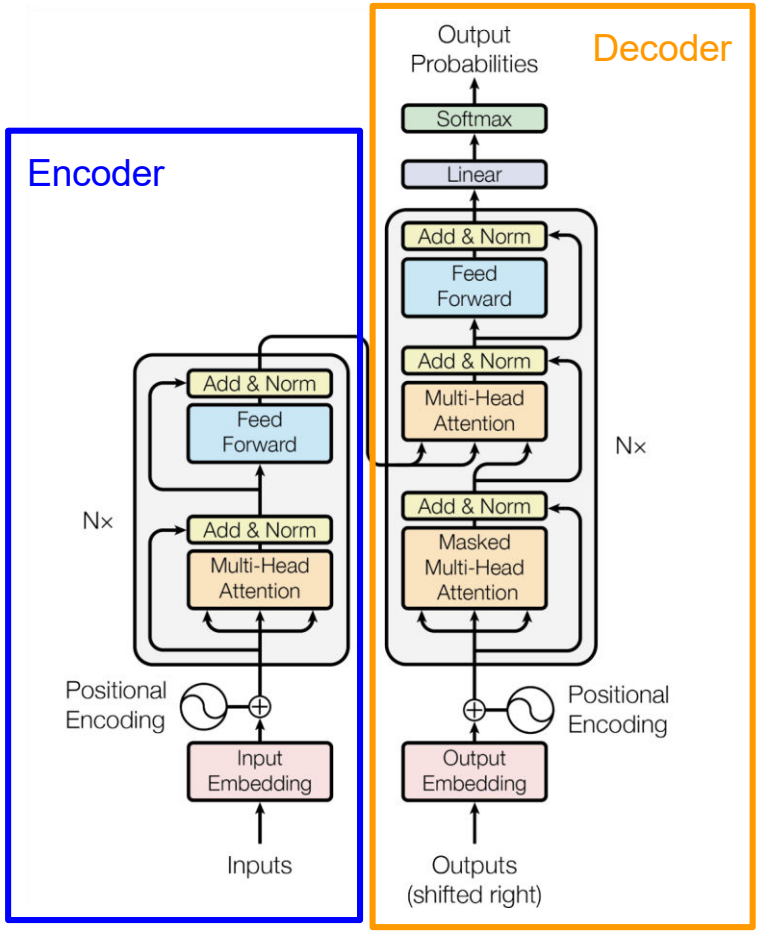
Transformer - Architecture Overview



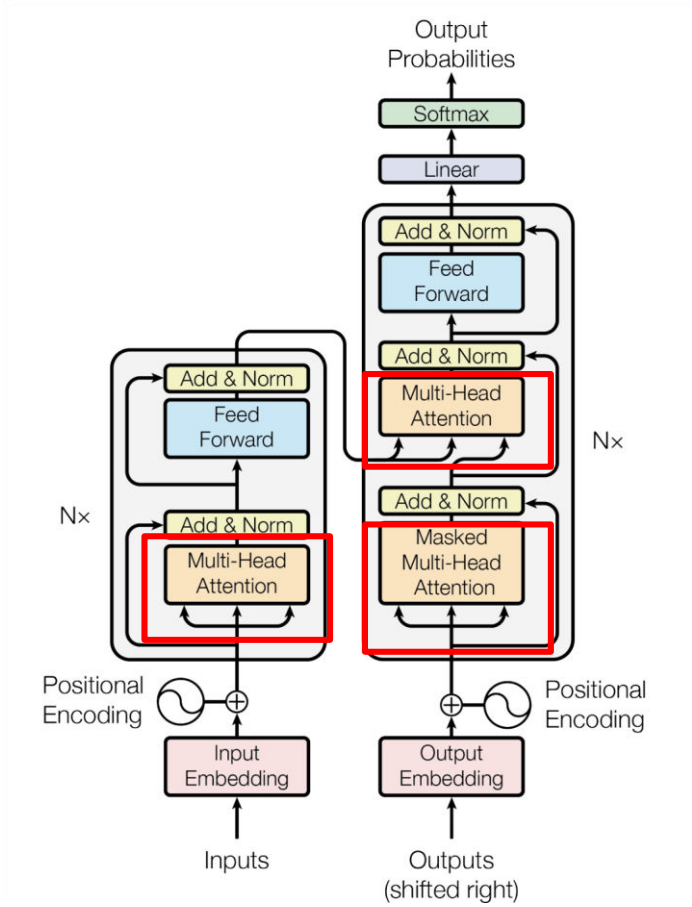
Transformer - Architecture Overview



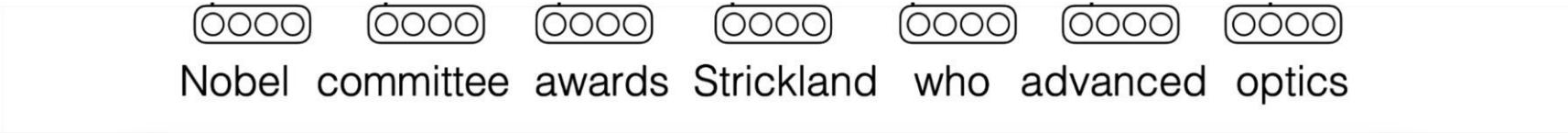
Transformer - Architecture Overview



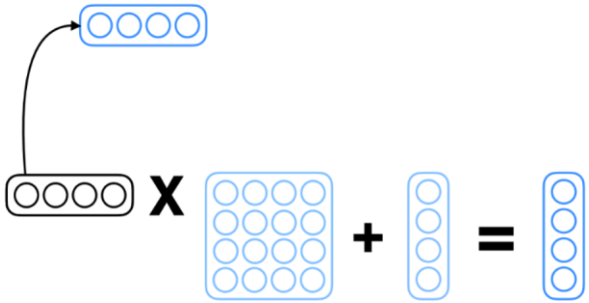
Transformer - Self Attention

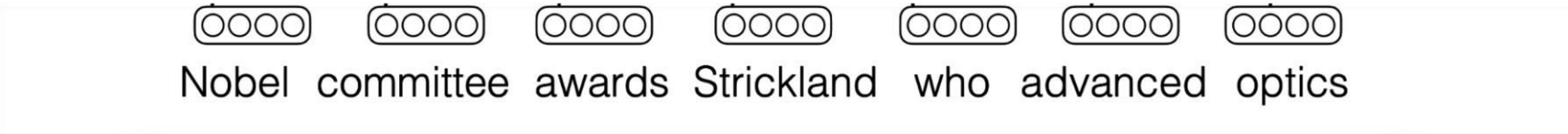


Transformer - Self Attention



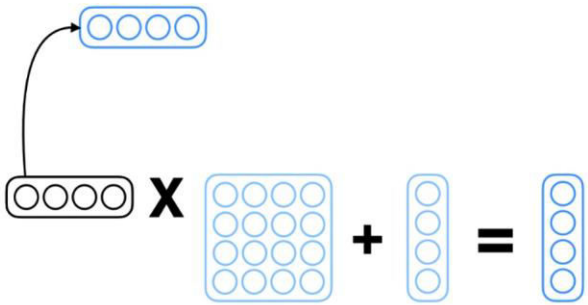
Transformer - Self Attention

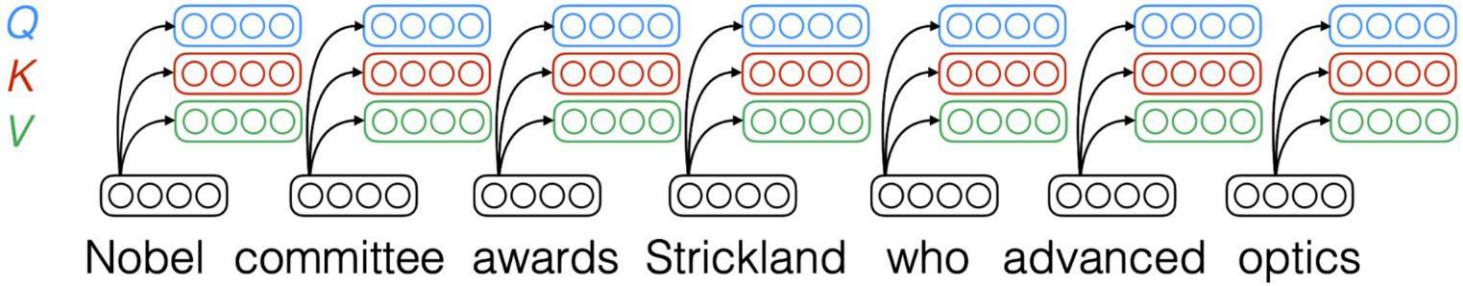




Transformer - Self Attention

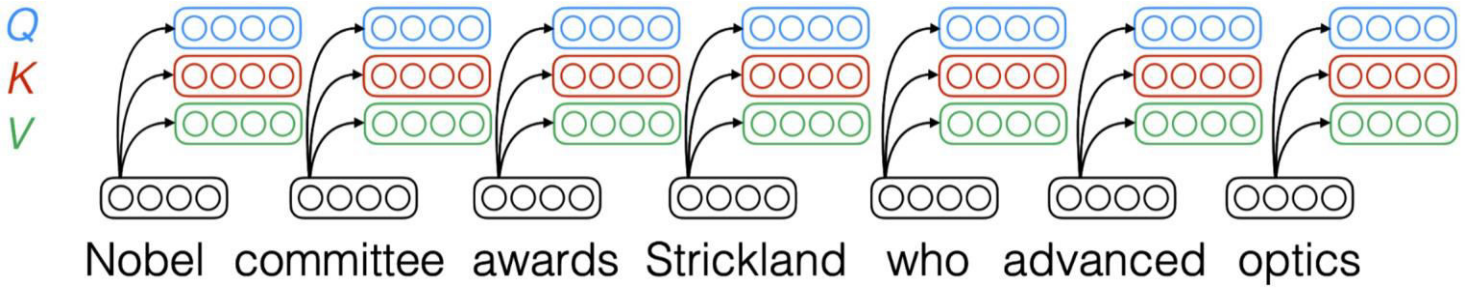
Q: Query
K: Key
V: Value





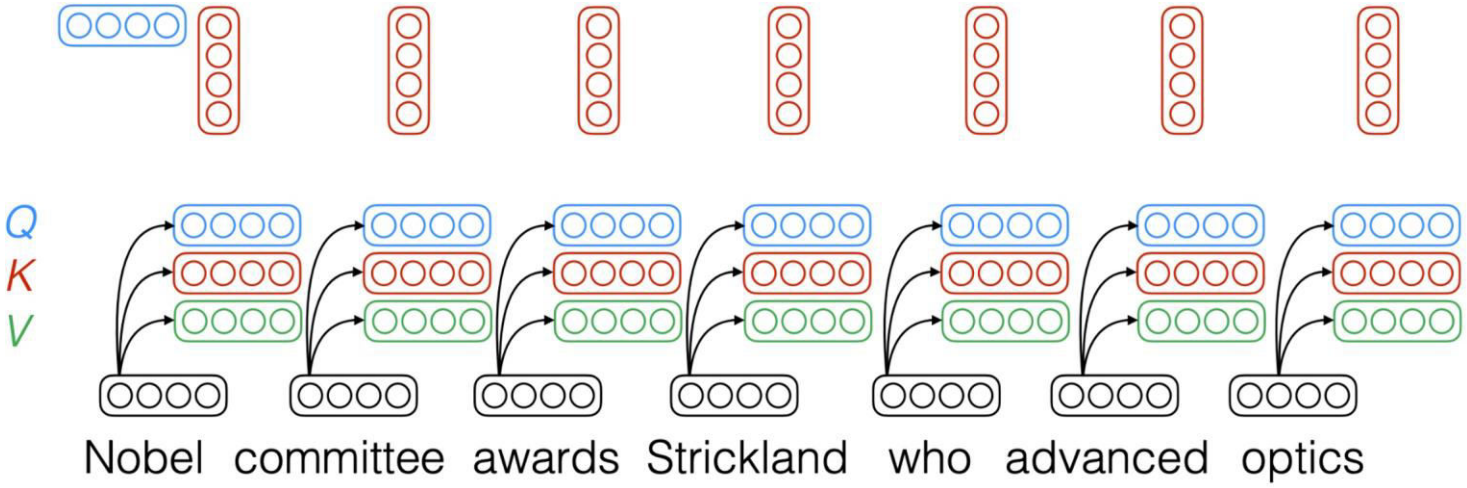
Transformer - Self Attention

Q: Query
K: Key
V: Value



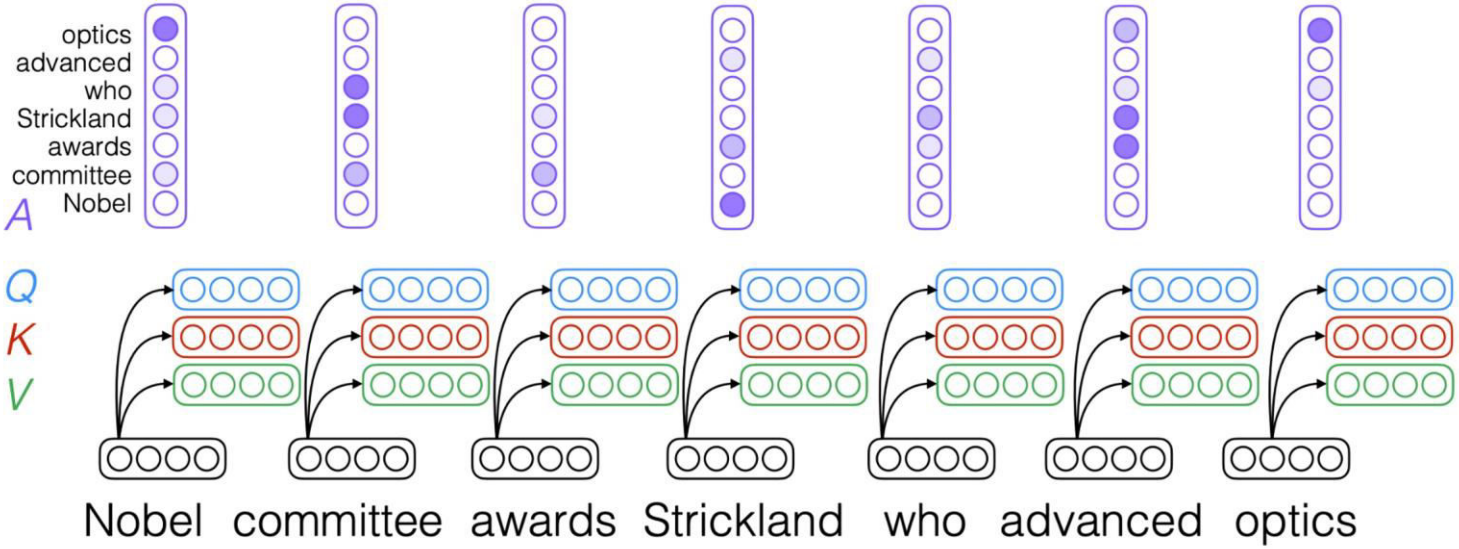
Transformer - Self Attention

Calculate Attention scores: between Q: Query and K: Key

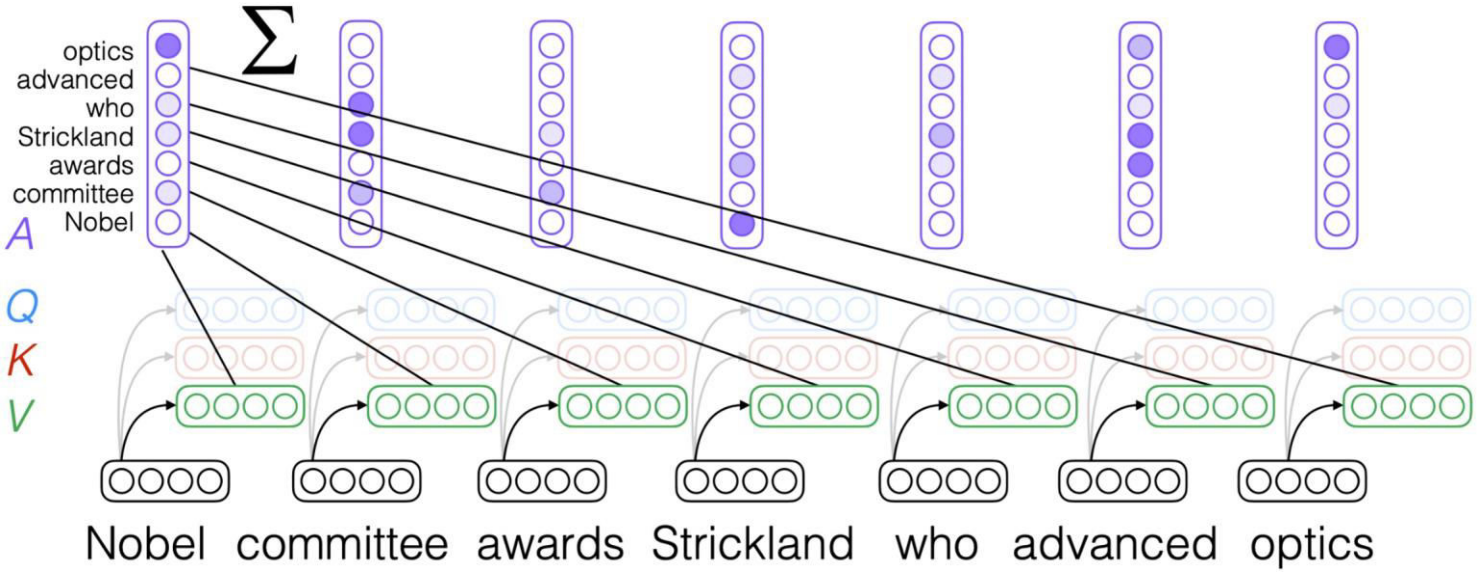


Transformer - Self Attention

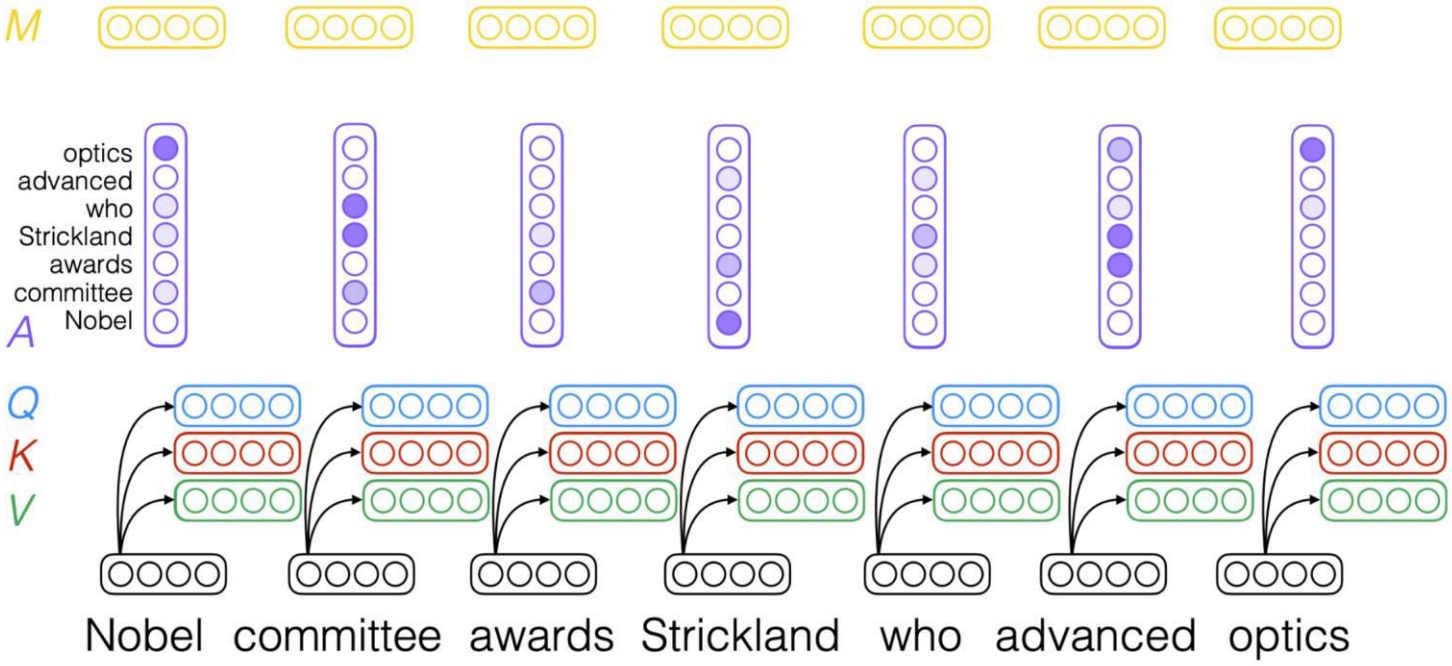
Each word is attended to all other words.



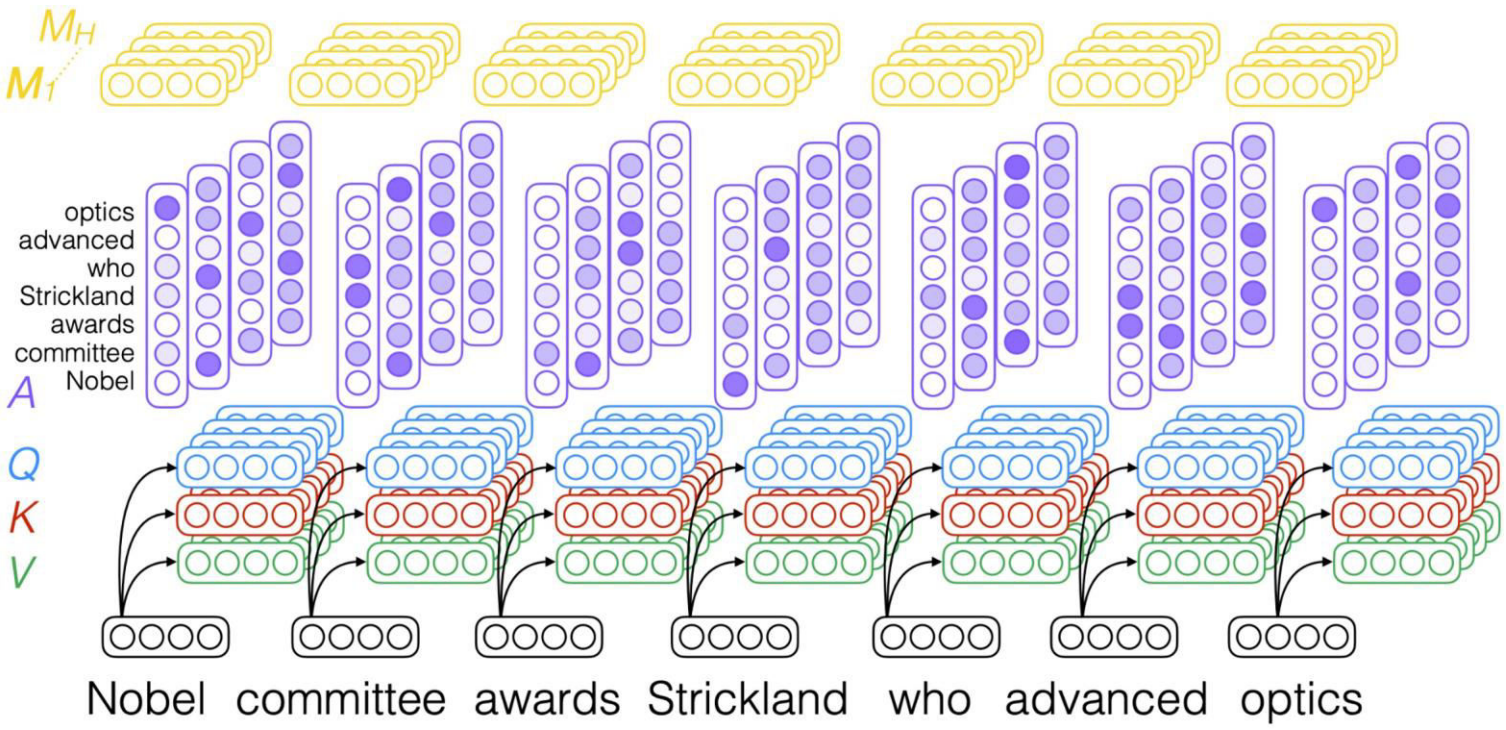
Transformer - Self Attention



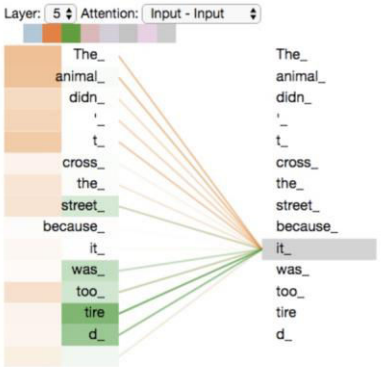
Transformer - Self Attention



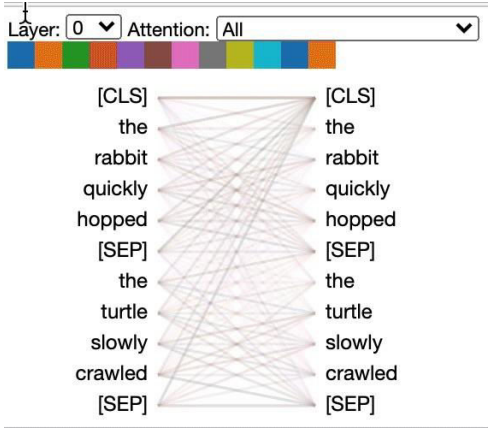
Transformer - Multi-Head Self Attention



Transformer - Multi-Head Self Attention



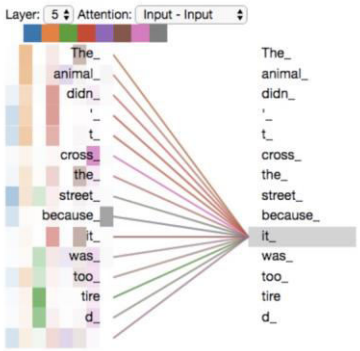
As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired".

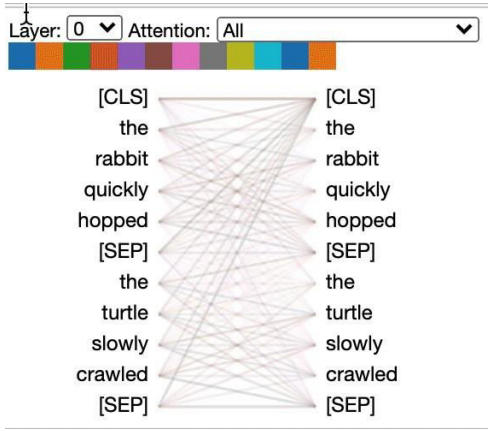


https://jalammar.github.io/illustrated-transformer/

Transformer - Multi-Head Self Attention

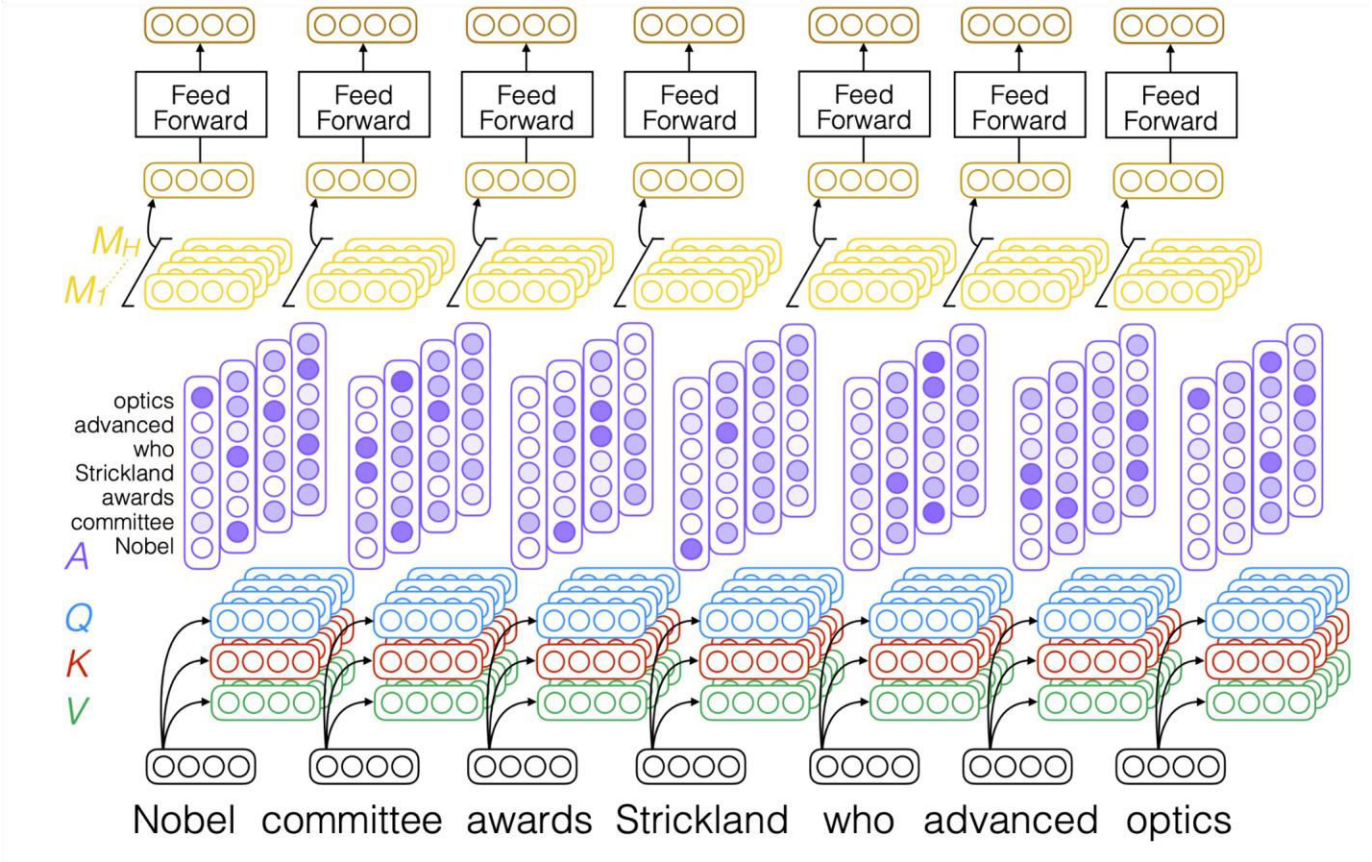
If we add all the attention heads to the picture, however, things can be harder to interpret:



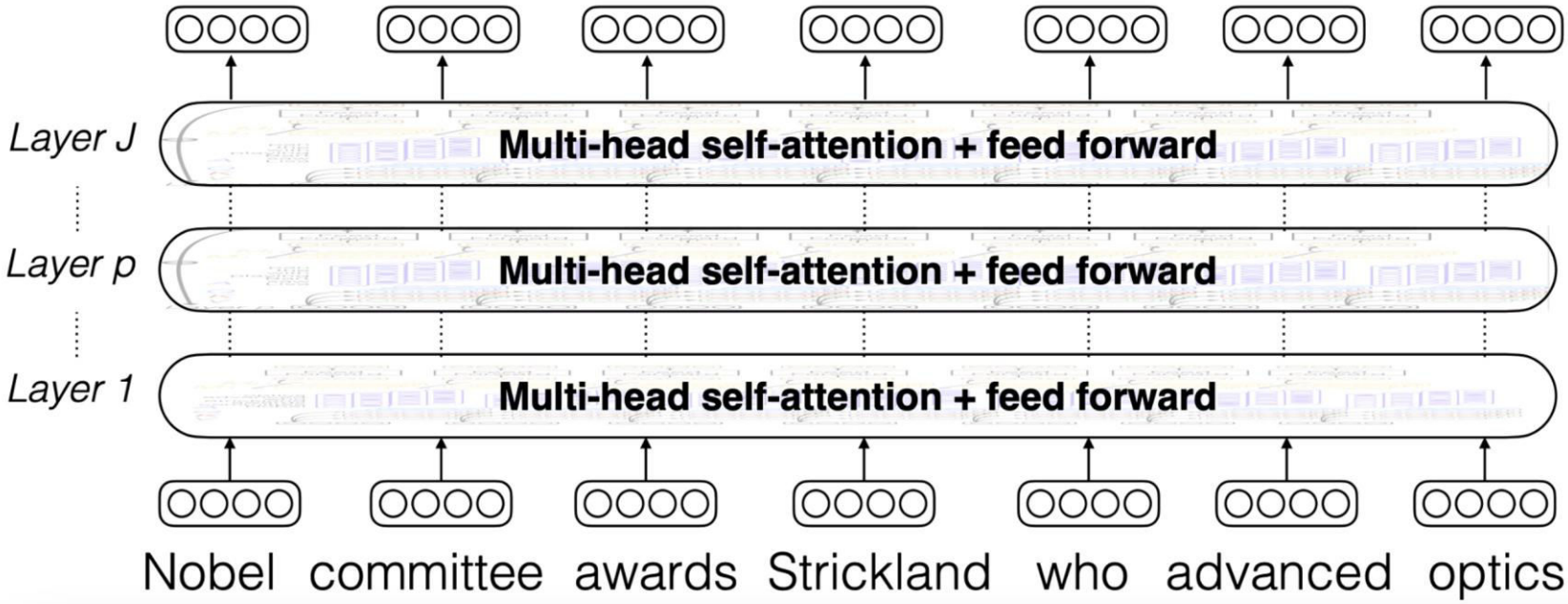


https://jalammar.github.io/illustrated-transformer/

Transformer Layer

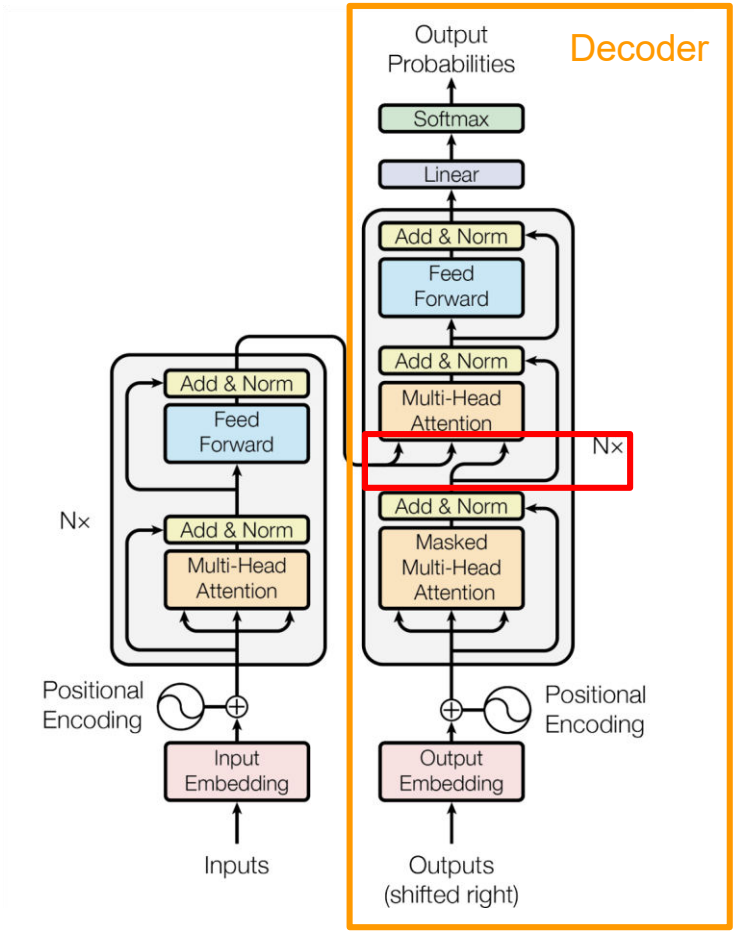


Transformer

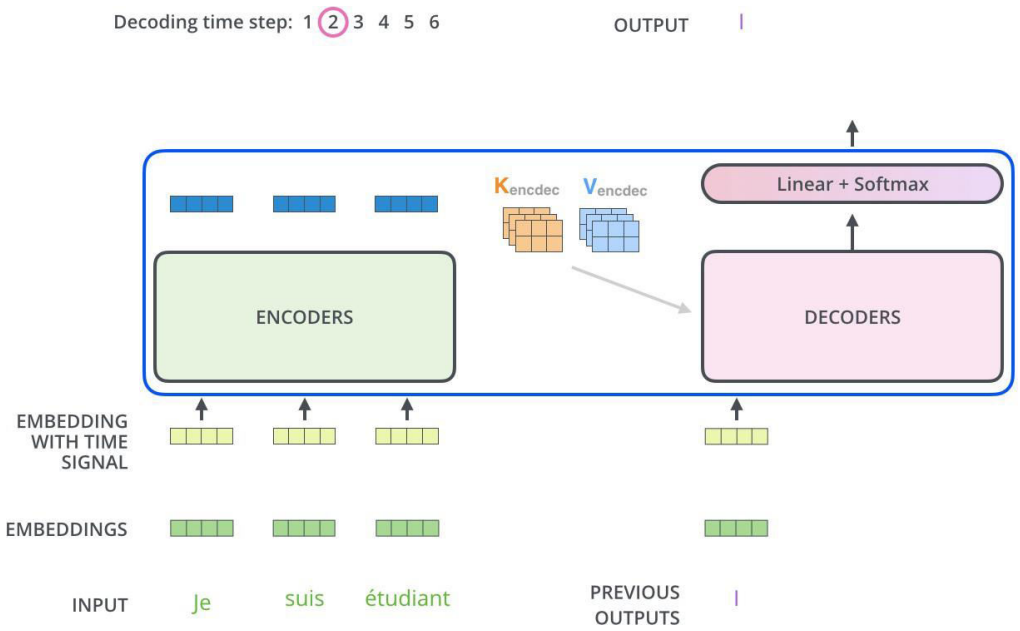


Transformer – decoder side

Transformer - Architecture Overview



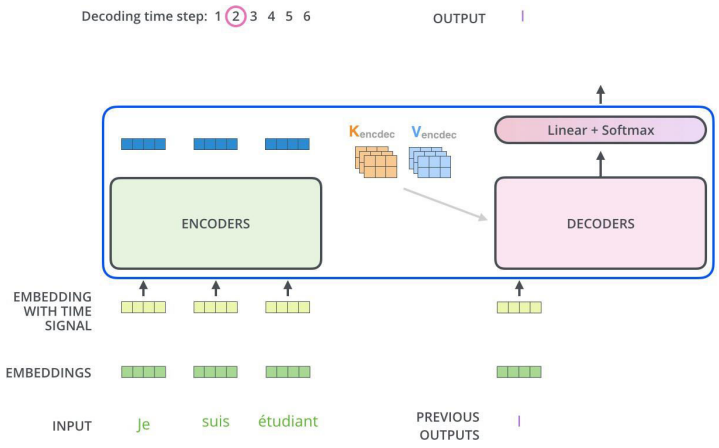
Transformer – decoder side



https://jalammar.github.io/illustrated-transformer/

Transformer – decoder side

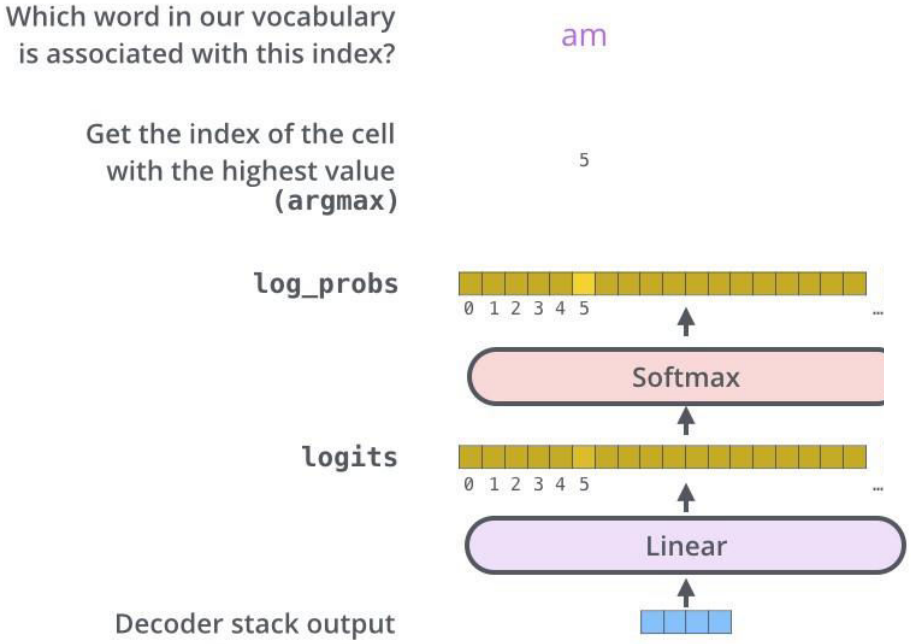
- The self attention layers in the decoder operate in a slightly different way than the one in the encoder:
  - In the decoder, the self-attention layer is only allowed to **attend to earlier positions in the output sequence**.
  - The “Encoder-Decoder Attention” layer works just like multiheaded self-attention, except it creates its Queries matrix from the layer below it, **and takes the Keys and Values matrix from the output of the encoder stack**.



https://jalammar.github.io/illustrated-transformer/

Transformer – decoder side

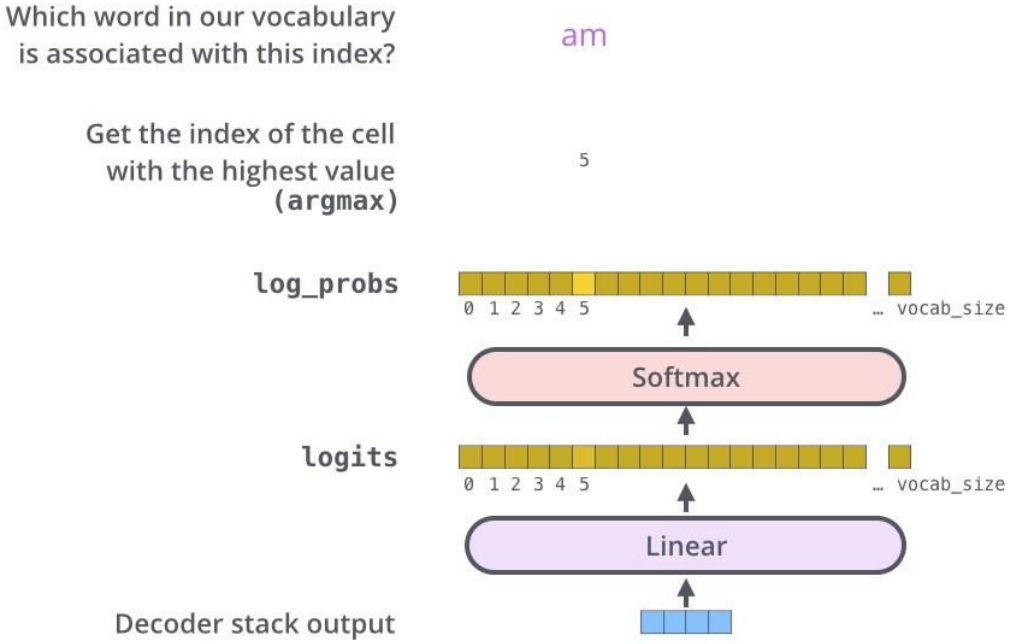
- The Final Linear and Softmax Layer



https://jalammar.github.io/illustrated-transformer/

Transformer – decoder side

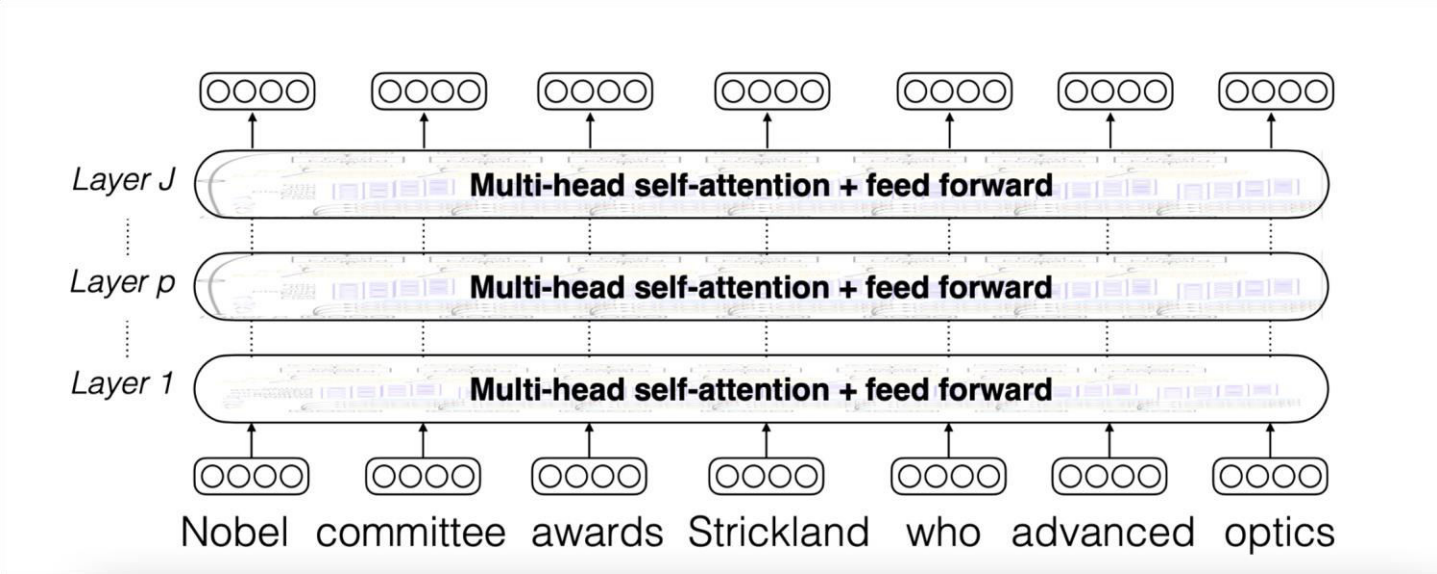
- The Final Linear and Softmax Layer



https://jalammar.github.io/illustrated-transformer/

Contextualized Word Embeddings

- Problems with word2vec/GloVe: single representation for each word.
- Contextualized Word Embeddings: representations depend on context.



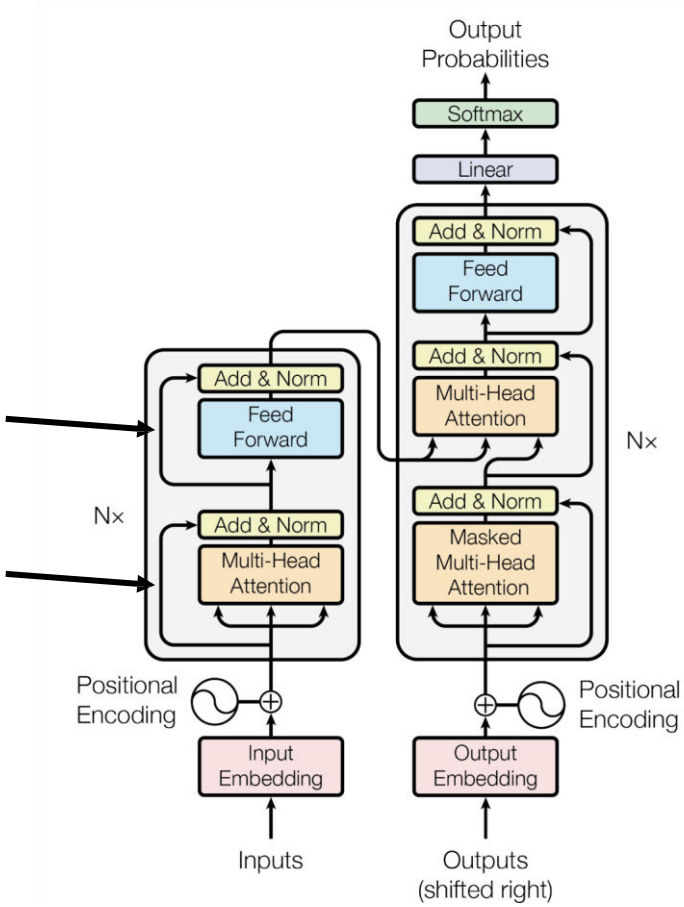
Other Components in Transformers

- Residual Connection
- Layer Normalization
- Masked Self Attention in Decoder
- Encoder-Decoder Cross Attention
- Sinusoidal Positional Encoding

Residual Connection

Residual Connection (He et al., 2016)

- Add input to its output



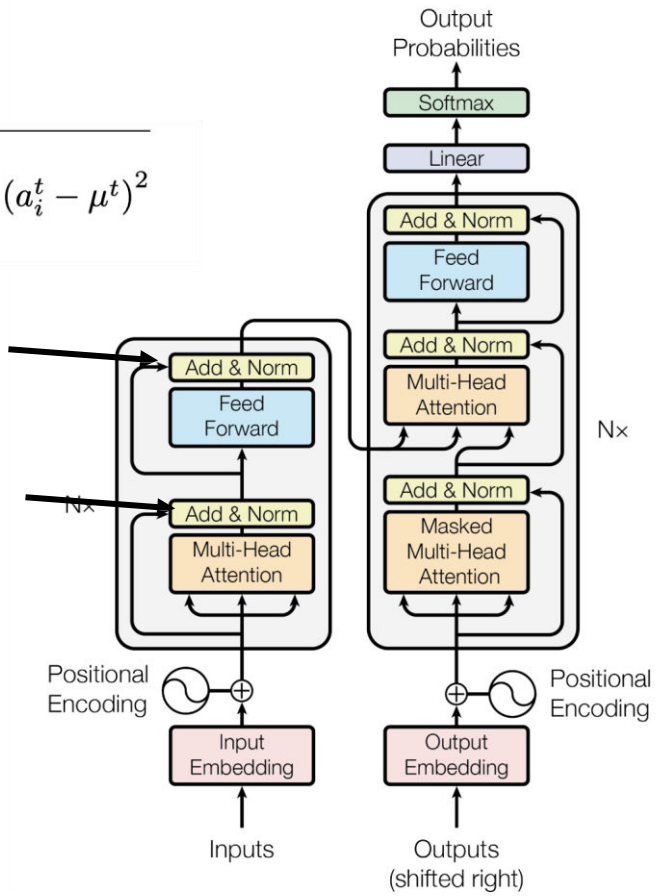
Layer Normalization

Layer Normalization (Ba et al., 2016)

$$\mathbf{h}^t = f\left[\frac{\mathbf{g}}{\sigma^t} \odot \left(\mathbf{a}^t - \mu^t\right) + \mathbf{b}\right] \qquad \mu^t = \frac{1}{H}\sum_{i=1}^{H} a_i^t \qquad \sigma^t = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^t - \mu^t\right)^2}$$

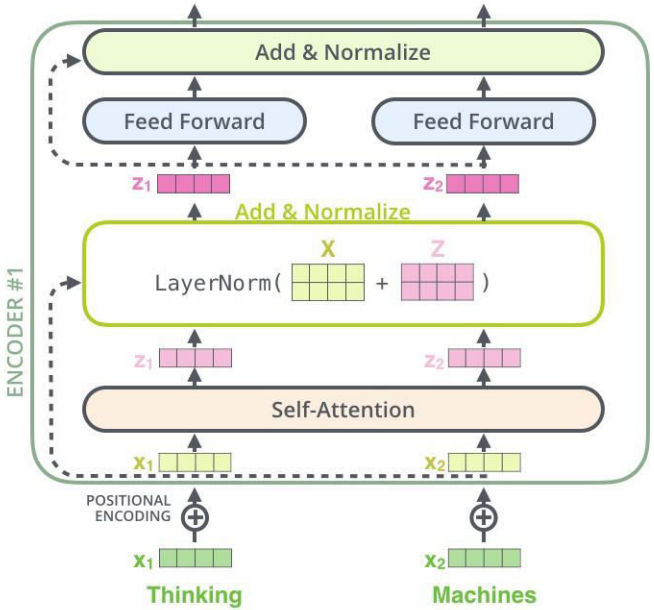
```python
class LayerNorm(nn.Module):
    "Construct a layernorm module (See citation for details)."
    def __init__(self, features, eps=1e-6):
        super(LayerNorm, self).__init__()
        self.a_2 = nn.Parameter(torch.ones(features))
        self.b_2 = nn.Parameter(torch.zeros(features))
        self.eps = eps

    def forward(self, x):
        mean = x.mean(-1, keepdim=True)
        std = x.std(-1, keepdim=True)
        return self.a_2 * (x - mean) / (std + self.eps) + self.b_2
```



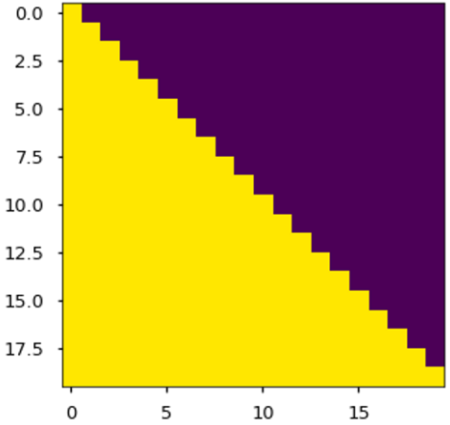
Residual Connection & Layer Normalization

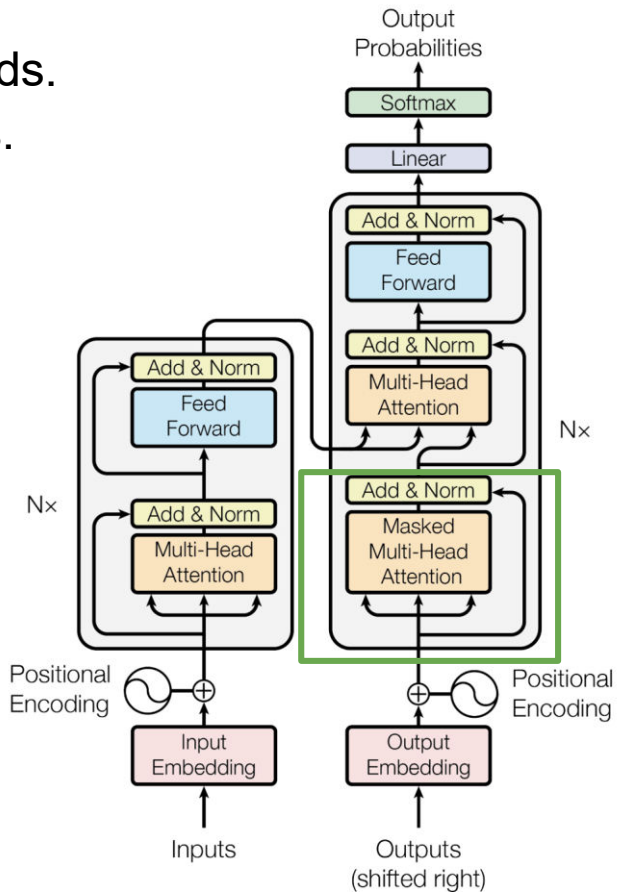
- To put them together



Masked Self Attention in Decoder

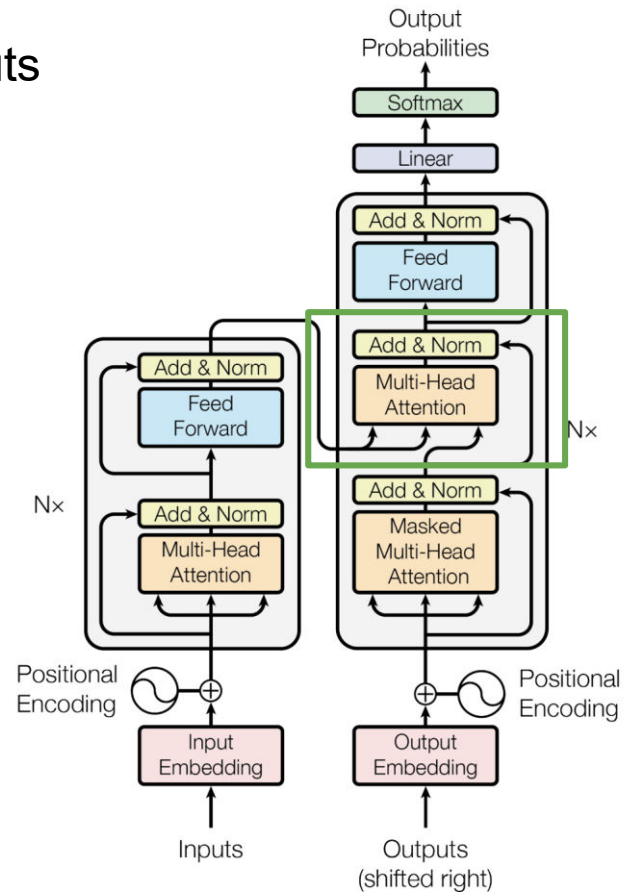
- In encoder, we can always look at all input words.
- In decoder, we can only look at previous words.
  - That means, we need to "**mask**" future words when performing attention.
  - Words are blocked for attending to future words during training.





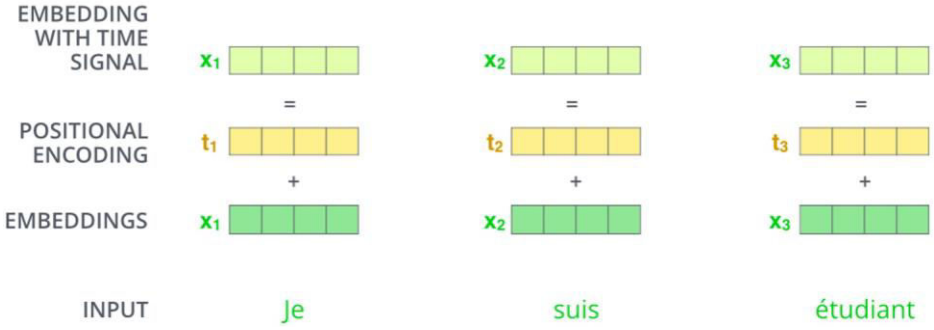
Encoder-Decoder Cross Attention

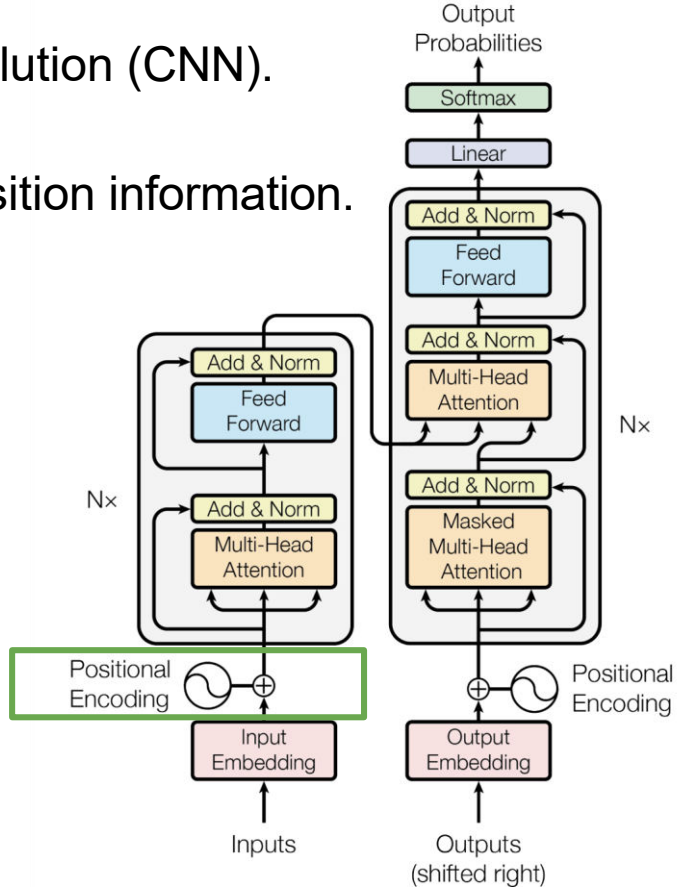
- Attention between encoder self-attention outputs and decoder masked self attention outputs.



Positional Encoding

- There is no recurrence (RNN) and no convolution (CNN).
- Self-attention do not have order information.
- So we need positional encoding to bring position information.





Sinusoidal Positional Encoding

- There are many different choices of positional encodings (Gehring et al., 2017)
  - some are **learned** (BERT, Devlin et al. 2018) and some are fixed
  - some for **relative** position and some for **absolute** position.
- In Transformer, we use **Sinusoidal position encodings**: fixed embeddings and can generalize to any sequence length.

Sinusoidal Positional Encoding

- There are many different choices of positional encodings (Gehring et al., 2017)
  - some are learned and some are fixed
  - some for relative position and some for absolute position.
- In Transformer, we use **Sinusoidal position encodings**: fixed embeddings and can generalize to any sequence length.
- Intuition: (4-dimensional position encodings for 16 positions)

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Sinusoidal Positional Encoding

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

4-dimensional position encodings for 16 positions

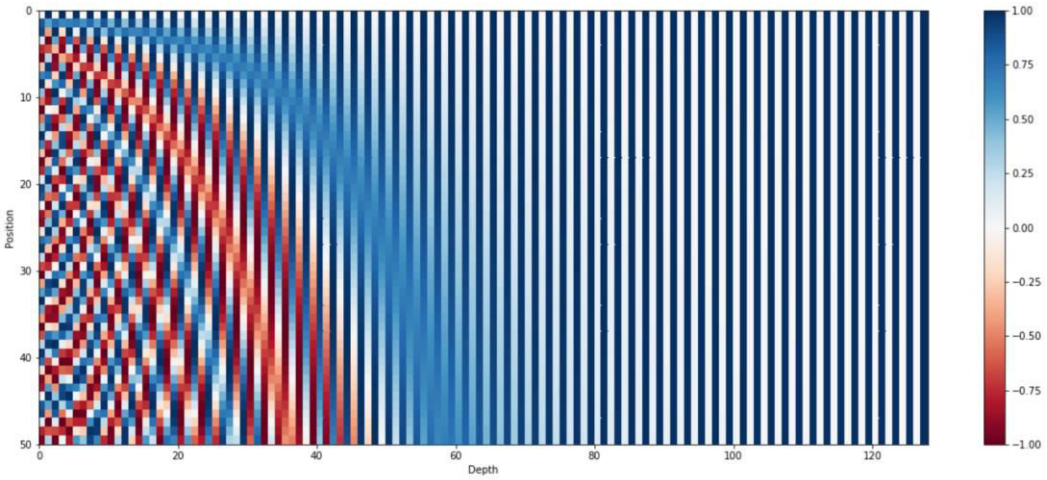
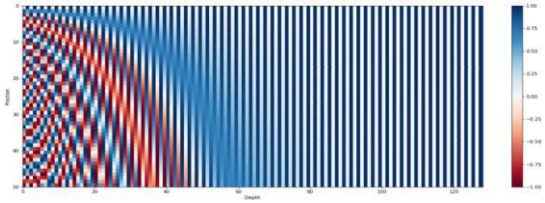


Figure 2 - The 128-dimensional positonal encoding for a sentence with the maximum lenght of 50. Each row represents the embedding vector $\overrightarrow{p_t}$

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Sinusoidal Positional Encoding



$$\overrightarrow{p_t}^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k . t), & \text{if } i = 2k \\ \cos(\omega_k . t), & \text{if } i = 2k+1 \end{cases} \quad \text{with} \quad \omega_k = \frac{1}{10000^{2k/d}}$$

t: position index

i: dimension index

d: total dimension

$$\overrightarrow{p_t} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_{d \times 1}$$

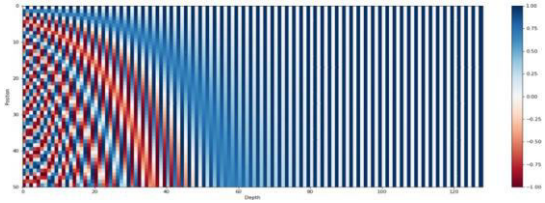
https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Sinusoidal Positional Encoding - Properties

- It should output a unique encoding for each time-step (word's position in a sentence)
- Distance between any two time-steps should be consistent across sentences with different lengths.
- Our model should generalize to longer sentences without any efforts. Its values should be bounded.
- It must be deterministic.

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Positional Encoding



- “Attention is all you need”: Our model should generalize to longer sentences without **any efforts**. Its values should be bounded. Deterministic

- There are many different choices of positional encodings (Gehring et al., 2017)
  - some are **learned** ?

Other Tricks in Transformers

- Subword Tokenization
- Optimizer
- Label Smoothing

OOV Issue

How to handle unknown words, i.e., a word at test time that we've never seen in our training data. It is also called OOV (out-of-vocabulary) words.

If we use word-level tokenization, we cannot have an index for unknown words, and we cannot have a word embedding for unknown words.

OOV Issue

Old solution: replace low-frequency words in training data with a special token <UNK>; then after training, we have a word embedding for <UNK>; if we have an unknown word during testing, we replace it with <UNK>.

New solution: **Subword tokenization**.

Subword Tokenization

Use Subword Units for tokenization.

Based on a simple algorithm called byte pair encoding (Gage, 1994).

First used successfully for NLP for machine translation by (Sennrich et al., 2016).

Byte Pair Encoding

Perform pre-tokenization

word	frequency
hug	10
pug	5
pun	12
bun	4
hugs	5

Form a base vocabulary: **b, g, h, n, p, s, u**

https://huggingface.co/transformers/tokenizer_summary.html

Byte Pair Encoding

Count all the character pair.

word	frequency
h+u+g	10
p+u+g	5
p+u+n	12
b+u+n	4
h+u+g+s	5

character pair	frequency
ug	
pu	
un	
hu	
gs	

https://huggingface.co/transformers/tokenizer_summary.html

Byte Pair Encoding

Count all the character pair.

word	frequency
h+u+g	10
p+u+g	5
p+u+n	12
b+u+n	4
h+u+g+s	5

character pair	frequency
ug	20
pu	17
un	16
hu	15
gs	5

https://huggingface.co/transformers/tokenizer_summary.html

Byte Pair Encoding

Merge the most frequent character pair, i.e., **ug,** and add it to the vocabulary. Then rotokenize and repeat this process.

The next to merge is ?

word	frequency
h+*ug*	10
p+*ug*	5
p+u+n	12
b+u+n	4
h+*ug*+s	5

https://huggingface.co/transformers/tokenizer_summary.html

Byte Pair Encoding

Merge the most frequent character pair, i.e., **ug,** and add it to the vocabulary. Then rotokenize and repeat this process.

The next to merge is **un**, so we add un to the vocabulary.

word	frequency
h+*ug*	10
p+*ug*	5
p+u+n	12
b+u+n	4
h+*ug*+s	5

character pair	frequency
un	16
h+ug	15
pu	12
p+ug	5
ug+s	5

https://huggingface.co/transformers/tokenizer_summary.html

Byte Pair Encoding

Repeat until some fixed number of merges, or until we reach a target vocab size.

word	frequency
hug	10
p+*ug*	5
p+*un*	12
b+*un*	4
hug + s	5

new vocab: **b, g, h, n, p, s, u, ug, un, hug**

https://huggingface.co/transformers/tokenizer_summary.html

Byte Pair Encoding

To avoid OOV, all possible characters / symbols need to be included in the base vocab.

- This can be huge if including all ***unicode*** characters (size 144,697)!
- GPT-2 uses ***bytes*** as the base vocabulary (size 256) and then applies BPE on top of this sequence (with some rules to prevent certain types of merges).
- GPT-2 has a vocabulary size of 50,257, which corresponds to the 256 bytes base tokens, a special end-of-text token and the symbols learned with 50,000 merges.

Commonly have vocabulary sizes of 32K to 64K

Encoding and Decoding

Encoding

- Sort the vocab from longest subwords to shortest subwords.
- Go through the vocab, and replace the longest subword first.
- This is very computationally expensive.
- In practice, we can cache how popular words are encoded.

Decoding

- In fact, how to determine word boundaries.

Encoding and Decoding

Encoding

- Sort the vocab from longest subwords to shortest subwords.
- Go through the vocab, and replace the longest subword first.
- This is very computationally expensive.
- In practice, we can cache how popular words are encoded.

Decoding

- In fact, we will add </w> to each word to indicate word boundaries.
- For example, if the encoded sequence is [“the</w>”, “high”, “est</w>”, “moun”, “tain</w>”], we immediately know the decoded sequence “the highest mountain</w>”.

Other subword Tokenization

Byte Pair Encoding

- merge by frequency
- used by GPT-2.

WordPiece (Schuster et al., 2012)

- merge by likelihood given by language models, not by frequency
- used by BERT.

SentencePiece (Kudo et al., 2018)

- All tokenization algorithms described so far have the same problem: It is assumed that the input text uses spaces to separate words. However, not all languages use spaces to separate words.
- SentencePiece can train subword models directly from raw sentences

Can we don't do tokenization at all?

Old solution: use <UNK>.

New solution: Subword tokenization

New New solution: tokenizer-free and directly operate on bytes!

- [CANINE: Pre-training an Efficient Tokenization-Free Encoder for Language Representation.](#) (Clark et al, 2022)
- [ByT5: Towards a token-free future with pre-trained byte-to-byte models.](#)

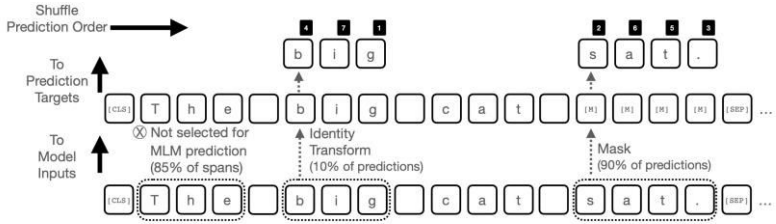


Figure 2: CANINE-C pre-training data preparation (§3.2.1). Character-wise predictions are made by an auto-regressive transformer layer that predicts then reveals one character at a time, in a shuffled order.

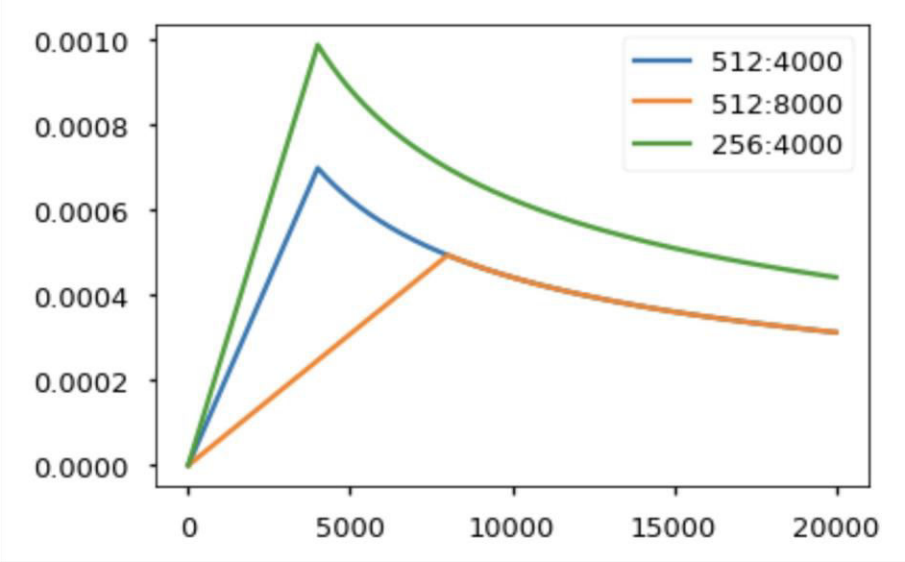
Optimizer

https://arxiv.org/abs/1412.6980

Adam Optimizer with Linear Warmup Learning Rate

We used the Adam optimizer (cite) with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula: $lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$ This corresponds to increasing the learning rate linearly for the first $warmup_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup_steps = 4000$.

Adam (Adaptive Moment Estimation) is a computationally efficient optimization mechanism for robustly training deep learning models with fast convergence. It calculates individual adaptive learning rates for different parameters, using estimates of first and second moments of the gradients.



- Adam

$$m_0 = 0, v_0 = 0$$

$$m_{t+1} \leftarrow \beta_1 m_t + (1 - \beta_1)\nabla_\theta \mathcal{L}(\theta)$$

Momentum

$$v_{t+1} \leftarrow \beta_2 v_t + (1 - \beta_2)\nabla_\theta \mathcal{L}(\theta)^2$$

RMS Prop

$$\theta_j \leftarrow \theta_j - \frac{\epsilon}{\sqrt{v_{t+1}} + 1e^{-5}} m_{t+1}$$

RMS Prop + Momentum

Algo of Adam.

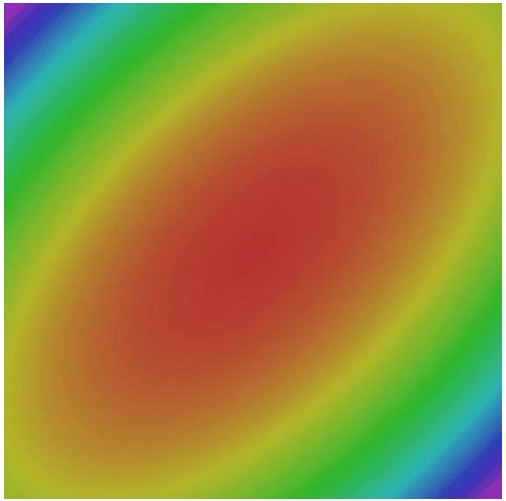


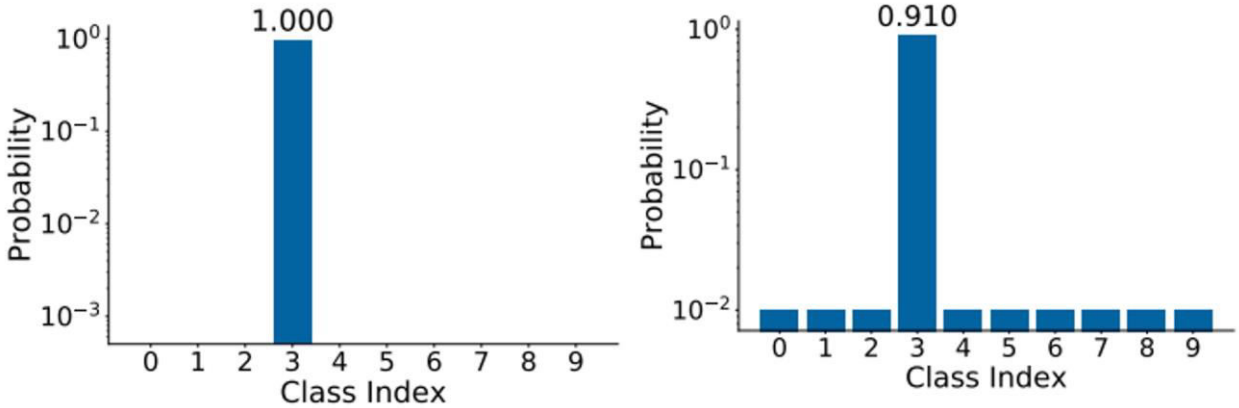
Fig. 6 Comparison of all optimization algorithms. (Visualization)

https://morioh.com/p/3f20600908f3

Label Smoothing

During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ (cite). This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

> *We implement label smoothing using the KL div loss. Instead of using a one-hot target distribution, we create a distribution that has* `confidence` *of the correct word and the rest of the* `smoothing` *mass distributed throughout the vocabulary.*



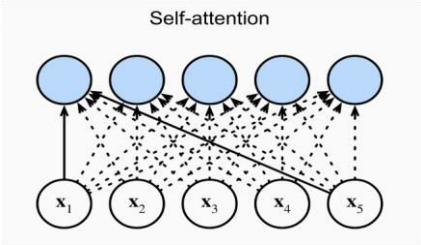
(a) Hard Label (b) LS

https://towardsdatascience.com/label-smoothing-make-your-model-less-over-confident-b12ea6f81a9a

Computational Complexity: Quadratic in Length!

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$\boxed{O(n^2 \cdot d)}$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$



Why Transformers

- Everything is attention (and FFNNs), you can train much larger networks if you have the right hardware (GPUs, TPUs) due to the improved parallelism. So much cheaper than RNNs.

- Downside: Only a few companies have the expertise and machine power (TPUs rather than GPUs) and money to build large language models.
  - GPT-2: 1.5B parameters
  - GPT-3: 175B parameters

Results

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.0**	$2.3 \cdot 10^{19}$	

Model Variations

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

Transformer-based Language Models

It is becoming the underlying architecture of most popular language models. Two most famous are:

- GPT: OpenAI Transformer-based Language Models
- BERT: Google Transformer-based Language Models
- RoBERTa: Facebook Transformer-based Language Models

Transformer beyond NLP

Transformer can also be used in many other applications: Vision, Speech, Reinforcement Learning, ...

People even consider we no longer needs CNN/RNN or any other neural nets, and Transformer is unifying many ML problems.

- Check Facebook's data2vec
- Check this Twitter post by Andrej Karpathy

> *"But as of approx. last two years, even the neural net architectures across all areas are starting to look identical - a Transformer (definable in ~200 lines of PyTorch github.com/karpathy/minGP…), with very minor differences. Either as a strong baseline or (often) state of the art."*